ForecastLabAI — end-to-end ML workflow

Problem

Answer "what happens technically when a user requests a demand forecast?" as a chain of responsibility.

Instant solution

Lifecycle (7 stages)

1. **Ingest** — store, product, sales, price, promotion, inventory records.
2. **Features** — lags, shifted rolling stats, calendar, price/promo, lifecycle signals.
3. **Train** — naive baselines, regression, tree-based forecasters.
4. **Backtest** — chronological folds + error metrics.
5. **Select & register** — compare candidates, store artifacts, set a champion alias.
6. **Forecast** — predict future demand, what-if scenarios.
7. **AI decision support** — RAG assistant + experiment agent.

On a forecast request

1. API (Pydantic) validates store, product, time range, model config.
2. Historical data is read from PostgreSQL.
3. Features are built with temporal boundaries enforced.
4. Forecasting service trains or loads the model and predicts.
5. Backtesting / model selection compare approaches on past "future" windows.
6. Run config, metrics and artifact metadata go to the registry (reproducibility).
7. Frontend shows predictions and KPIs; agents can fetch evidence or run experiments.

Why it works / gotchas

- These are **separate API workflows**, not one automatic request. Say so.
- The repo wires them together in the demo: `make demo` → `scripts/run_demo.py` (in-process version: `app/features/demo/pipeline.py`): `seed → features → train ×3 → backtest → register → alias → agent`.
- Jobs run synchronously behind async-ready API contracts (`app/features/jobs/service.py`); model selection uses an in-process background task (`asyncio.create_task`). Not a durable queue.

Diagram brief

- Type: left-to-right flowchart, 7 numbered stage boxes, stage 7 visually different (AI layer).
- Under each stage: the module that owns it (ingest, featuresets, forecasting, backtesting, model_selection + registry, forecasting + scenarios, rag + agents).
- Second lane (optional): the demo sequence seed → features → train ×3 → backtest → register → alias → agent.
- Feedback edge: registry → forecasting labelled "load champion via alias".
- Do NOT draw a message queue or workers.